

Synthèse Grafana

Couche de visualisation par-dessus Prometheus, pour le projet **DevOps Portfolio MERN**. Data source, dashboards, panneaux, options de visualisation, et industrialisation en « dashboards as code ».

Suite de

la synthèse Prometheus

Cluster

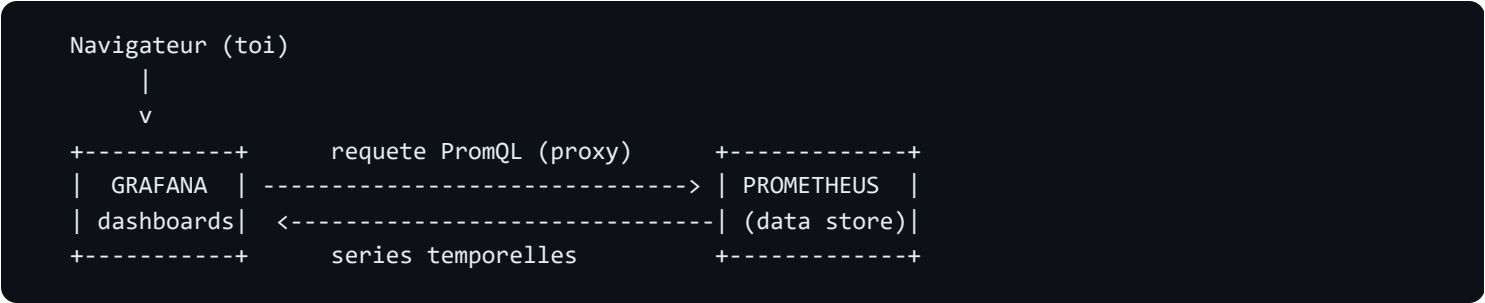
Kubernetes local (kind)

Namespace

monitoring

1. Rôle de Grafana

Grafana ne stocke **aucune** métrique. C'est un outil de **visualisation** qui se connecte à des **sources de données** (ici Prometheus), les interroge en PromQL, et affiche le résultat sous forme de **tableaux de bord** composés de **panneaux**.



2. Concepts clés

Concept	Définition
Data source	La source que Grafana interroge (Prometheus, Loki...). Mode proxy = Grafana l'appelle côté serveur.
Dashboard	Une page composée de panneaux. Techniquement, un simple document JSON .
Panel (panneau)	Une visualisation d'une ou plusieurs requêtes PromQL.
Type de visualisation	La forme du panneau : <i>Time series</i> (courbe), <i>Gauge</i> (cadran), <i>Stat</i> (gros chiffre), <i>Table</i> ...
Variable de dashboard	Menu déroulant en haut du dashboard (ex. job , instance) pour filtrer dynamiquement et rendre le dashboard réutilisable.
Provisioning	Configurer Grafana « en code » au démarrage (data sources et dashboards via des fichiers), au lieu de cliquer dans l'UI.

3. Composants déployés

Élément	Rôle	Fichier
Data source provisionnée	Prometheus déclaré en code (UID fixe <code>prometheus</code>)	<code>grafana/01-datasource.yaml</code>
Deployment	Le serveur Grafana (image, admin, montages)	<code>grafana/02-deployment.yaml</code>
Service	Accès interne (port 3000)	<code>grafana/03-service.yaml</code>
Provider de dashboards	Dit à Grafana de charger les JSON d'un dossier	<code>grafana/04-dashboard-provider.yaml</code>
Dashboard « DevOps Portfolio — Application »	Le tableau de bord versionné en JSON	<code>grafana/dashboards/devops-portfolio.json</code>

4. Le dashboard « DevOps Portfolio — Application »

Le tableau de bord a été **recentré sur l'application et le métier** plutôt que sur l'infrastructure : les métriques CPU/RAM de la machine vivent désormais dans le dashboard communautaire *Node Exporter Full* (ID 1860), dédié à cela. Ici on suit la **santé du service** (méthode RED : Rate, Errors, Duration) et des **indicateurs métier**.

Panneaux d'état (Stat)

Panneau	Sens	Requête PromQL
Backend en ligne	Le backend répond-il au scrape ?	<code>max(up{job="backend-app"})</code>
Projets en base (réel)	Nombre réel de projets stockés (jauge)	<code>max(portfolio_projects_count)</code>
Connexions réussies	Cumul des logins OK	<code>sum(auth_login_attempts_total{result="success"})</code>
Connexions échouées	Indicateur de sécurité (brute-force)	<code>sum(auth_login_attempts_total{result=~"failure error"})</code>
Taux de succès des connexions (%)	Part de logins réussis sur le total	<code>100 * sum(auth_login_attempts_total{result="success"}) / sum(auth_login_attempts_total)</code>
Projets créés (cumul événements)	Compteur d'événements depuis le démarrage du pod	<code>sum(projects_created_total)</code>
Redémarrages de pods (cumul)	Détecte les crash-loops (ex. OOM)	<code>sum(kube_pod_container_status_restarts_total{namespace="devops-portfolio-dev"})</code>

Panneaux temporels (Time series)

Panneau	Requête PromQL
Trafic — requêtes/s par route	<code>sum(rate(http_requests_total[5m])) by (route)</code>
Latence p95 par route (s)	<code>histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le, route))</code>
Taux d'erreurs 5xx (%)	<code>100 * (sum(rate(http_requests_total{status_code=~"5.."}[5m])) / sum(rate(http_requests_total[5m])))</code>
Connexions par résultat (/s)	<code>sum(rate(auth_login_attempts_total[5m])) by (result)</code>
Redémarrages de pods par pod	<code>sum(kube_pod_container_status_restarts_total{namespace="devops-portfolio-dev"}) by (pod)</code>

Counter vs Gauge — la nuance qui compte. « Projets créés (cumul) » est un **Counter** : il ne fait qu'augmenter et *repart à zéro si le pod redémarre*. « Projets en base (réel) » est une **Gauge** alimentée par un `countDocuments()` MongoDB rafraîchi à chaque scrape (hook `collect()` de `prom-client`) : elle reflète toujours l'**état réel** de la base et survit aux redémarrages. On garde les deux car ils racontent des choses différentes : un flux d'événements d'un côté, un stock de l'autre.

Pourquoi un panneau affiche « No data » ? Une série Prometheus n'existe qu'à partir de son premier point. Tant qu'aucune connexion réussie n'a eu lieu, `auth_login_attempts_total{result="success"}` n'existe pas → le panneau « Connexions réussies » affiche *No data* (et non 0). Dès le premier succès, la série apparaît.

5. Dashboards as code (la bonne pratique)

Plutôt que de créer les dashboards à la main (perdus si le pod redémarre), on les **versionne en JSON dans Git** et Grafana les charge automatiquement via le provider.

```
# Régénérer le ConfigMap des dashboards à partir des fichiers JSON
kubectl create configmap grafana-dashboards -n monitoring \
  --from-file=monitoring/grafana/dashboards \
  --dry-run=client -o yaml | kubectl apply -f -

kubectl rollout restart deployment/grafana -n monitoring
```

Piège d'encodage (Windows/PowerShell). Le pipe `... -o yaml | kubectl apply` peut corrompre les accents UTF-8 (ils s'affichent en « ??? » dans Grafana). Sous PowerShell, préférer une création directe sans pipe : `kubectl delete configmap grafana-dashboards -n monitoring --ignore-not-found ; kubectl create configmap grafana-dashboards -n monitoring --from-file=monitoring/grafana/dashboards` . En Bash, le pipe est sûr (flux d'octets).

Importer un dashboard communautaire (ex. « Node Exporter Full », ID `1860`) : UI → Dashboards → New → Import → saisir l'ID. Pour le versionner ensuite, déposer son JSON dans le dossier `dashboards/` et régénérer le ConfigMap.

6. Commandes utiles

```
# Accéder à l'UI Grafana (cluster kind => port-forward)
kubectl port-forward -n monitoring svc/grafana 3001:3000
# -> http://localhost:3001 (admin / admin)

# Réinitialiser le mot de passe admin (pas d'email de reset : SMTP non configuré pour Grafana)
kubectl exec -n monitoring deploy/grafana -- grafana cli admin reset-admin-password admin
```

7. Pièges rencontrés & bonnes pratiques

Piège	Leçon
Pas de sélecteur de data source à l'import	Normal si une seule source par défaut : Grafana la lie automatiquement.
Pod Grafana bloqué en <code>Pending</code>	Cluster mono-node à court de CPU + stratégie RollingUpdate (besoin du double). Passer en <code>strategy: Recreate</code> .
Mot de passe oublié, pas d'email	Grafana n'a pas de SMTP (seul Alertmanager en a). Reset via <code>grafana cli</code> .
Mot de passe / dashboards perdus au redémarrage	Stockage éphémère. Le mot de passe redevient celui de <code>GF_SECURITY_ADMIN_PASSWORD</code> . Pour persister : un PVC.
Dashboard provisionné « datasource not found »	Fixer un UID stable sur la data source (<code>uid: prometheus</code>) et le référencer dans le JSON.
Accents en « ??? » dans un panneau	ConfigMap recréé via un pipe PowerShell. Recréer sans pipe (voir §5).
Panneau « No data » alors qu'on attend 0	La série n'existe pas encore (counter jamais incrémenté). Normal jusqu'au premier événement.
Métrique métier absente après modif du code	Le pod tourne l'ancienne image. Reconstruire → pousser → <code>kubectl rollout restart</code> .

8. Glossaire express

Terme	En une phrase
Data source	Source de données interrogée par Grafana.
Panel	Un bloc de visualisation dans un dashboard.
Provisioning	Configuration de Grafana via des fichiers, au démarrage.
Counter	Compteur qui ne fait qu'augmenter (remis à zéro au redémarrage du process).
Gauge	Mesure instantanée qui monte et descend (ex. nombre de projets en base).
Recreate	Stratégie de déploiement : arrêter l'ancien pod avant de créer le nouveau.
PVC	PersistentVolumeClaim : stockage persistant (survit aux redémarrages).

9. Prochaines étapes

- **Persistence** : ajouter un PVC à Grafana (et à Prometheus) pour conserver état et historique.
- **Alerting Grafana** : créer des alertes directement depuis les panneaux + contact points (en complément d'Alertmanager).
- **Plus de dashboards as code** : versionner le 1860 et d'autres dashboards métier.
- **Industrialisation** : déployer la stack via Terraform (provider Helm) pour rester cohérent avec l'IaC du projet.